

天津大学

《软件工程中级实践》实验报告

题目：虚拟钱包



学 院 软件学院
专 业 软件工程
年 级 2023级
小 组 第04小组

学号	姓名
3023208045	高灿
3023244158	曾意
3023244168	贾思韵
3023244157	戴茗静
3023244362	杨晓越

2025 年 11 月 14 日

目 录

第一章 模块需求说明	1
1.1 引言	1
1.2 需求分析	2
1.3 业务分析	9
第二章 模块设计	17
2.1 数据库设计	17
2.2 后端接口设计	19
2.3 前端设计	22
第三章 模块实现	27
3.1 模块概览	27
3.2 业务逻辑实现	30
第四章 模块测试	34
4.1 测试概览	34
4.2 前端测试	34
4.3 后端测试	37
4.4 前后端联合测试	41
4.5 边界条件测试	44
4.6 异常场景测试	44
第五章 问题与解决&结果反思	46
5.1 问题与解决	46
5.2 结果反思	47
第六章 实践总结与展望	48
6.1 实践总结	48
6.2 未来展望	48

第一章 模块需求说明

1.1 引言

1.1.1 简介

本文档面向“饿了么”APP项目的客户决策者、项目经理、设计与开发团队、测试人员及相关参与方，旨在系统阐述产品新增的“虚拟钱包”核心功能需求。

“饿了么”APP项目致力于提供高效、便捷的在线订餐和商户管理服务。在第一版设计中，系统支付流程依赖于外部银行卡或第三方支付接口（如支付宝、微信支付）直连完成。

为满足平台对资金的精细化管理需求、支持创新的营销策略（如充值返点）及复杂的交易场景（如资金冻结、透支），本项目新增“虚拟钱包”功能。

“虚拟钱包”的实现将显著提升平台资金流转的灵活性和安全性，强化用户资金沉淀，并为未来业务扩展（如小额信贷、平台收款）奠定坚实基础。

1.1.2 背景

(1) 虚拟钱包的必要性

大部分成熟的电商和外卖平台均需建立内部的资金管理体系，即“虚拟钱包”功能，以实现资金在平台内部的闭环流转与控制。原系统直接通过第三方支付完成交易，无法实现资金的平台内暂存或精细化操作，导致以下核心业务和用户体验受限：

- a. 缺乏资金激励机制：无法支持如充值返点或奖励金等策略，难以有效鼓励用户在平台上沉淀资金
- b. 退款流程受限：退款流程需原路返回，耗时且无法暂存在用户账户内，影响用户体验
- c. 交易安全与风控：不具备资金冻结能力，难以应对延迟收货确认等需要中间担保的交易场景
- d. 金融服务缺失：不支持透支（小额信贷）和商户平台收款等增值金融服务

(2) 项目核心价值提升

虚拟钱包作为核心资金中介，用于管理用户在平台内的可支配资金，其增加将带来以下核心价值：

- a. 实现资金闭环管理：平台对资金流有完全的控制权，支持更复杂的业务逻辑（如内部转账、收款）
- b. 强化营销与激励：通过可配置的充值奖励规则，有效吸引用户充值，提升资金沉淀率
- c. 提高交易灵活性：支持退款暂存、资金冻结和透支等高级功能，全面优化交易流程的安全性 with 用户体验

1.1.3 定义、缩略语

表 1-1 定义与缩略语

术语	解释
钱包	指平台的虚拟钱包功能，用户可以在饿了么平台上使用通过钱包来使用虚拟资金进行交易
交易	用户利用平台的钱包产生的资金流动行为定义为交易，包括充值、提现、支付、收款和退款；交易状态分为正常和冻结，当资金在途时，交易状态显示冻结，其余情况为正常
贷款	仅用于消费时候，当钱包中的钱不够的时候，用户可以选择使用虚拟钱包贷款功能，利用透支来支付当前消费费用。
会员	用户可以通过支付会员费用成为不同等级的VIP用户，等级越高的会员可用的透支额度越高

1.2 需求分析

1.2.1 目标

(1) 核心目标

- 1.资金沉淀与激励：实现充值、提现功能，并通过可配置的奖励机制，有效激励用户在平台内沉淀资金。
- 2.交易原子性保证：确保支付、转账等关键交易流程具备原子性，保证资金流转的准确性和一致性。
- 3.高级金融服务：引入资金冻结、透支等功能，支持复杂的交易场景和对高价值用户的差异化服务。

(2) 采用第一种方案实现交易流水

通过将交易类型分为充值、提现和支付，同时入账钱包-出账钱包的交易形式，优化了交易流水的管理。



图 1-1 交易流水实现方案

(3) 在具体实现上使用 DDD充血模型

在模块的具体实现上，钱包功能采用领域驱动设计 DDD 中的充血模型 (Rich Domain Model)。

实现方式：钱包的核心业务逻辑，如计算充值奖励、处理提现扣费、执行转账的原子操作等，将内聚并封装在钱包领域模型之中。**Service** 层仅负责协调领域模型和外部调用，以及完成领域模型与数据库持久化模型（如 DAO/ORM）之间的转换。

优势：相比传统的贫血模型（Anemic Model，业务逻辑集中在 **Service** 层），充血模型增强了业务逻辑的封装性、一致性和可维护性。它确保了钱包对象在任何操作中都能维护自身的状态和业务规则，减少了业务逻辑泄露和服务层代码的冗余。

1.2.2 主要功能



图 1-2 虚拟钱包页面

1.核心功能

(1) 资金出入管理

充值：用户通过第三方支付平台（如微信支付、支付宝）将资金转入虚拟钱包。系统实时计入用户钱包余额，并根据规则发放奖励金；

充值奖励： 充值金额的 1% 作为平台赠送的奖励金。



图 1-3 虚拟钱包充值页面



图 1-4 虚拟钱包提现页面

提现：用户发起提现申请，将虚拟钱包中可用余额转出至绑定的第三方支付账户。系统在执行转账前将扣除相应手续费；

提现手续费： 提现金额的 5% 作为手续费扣除。

(2) 交易与流转

使用钱包消费：用户在支付时可直接使用钱包消费，用户与商家之间实现钱包对钱包的资金流转，此过程需严格保证交易的原子性。状态要求为资金实时扣除，实时入账。

退款暂存：订单取消或发生售后退款时，退款金额不会原路径返回，而是实时存入用户的虚拟钱包，并立即转为可用余额，可用于后续消费。实时到账，可用。提升退款体验。

(3) 会员与信用服务

会员升级：用户支付会员费用以获得不同等级的 VIP 身份。会员等级是透支额度的关键决定因素。等级越高的 VIP 用户，系统授予的可透支额度越高。

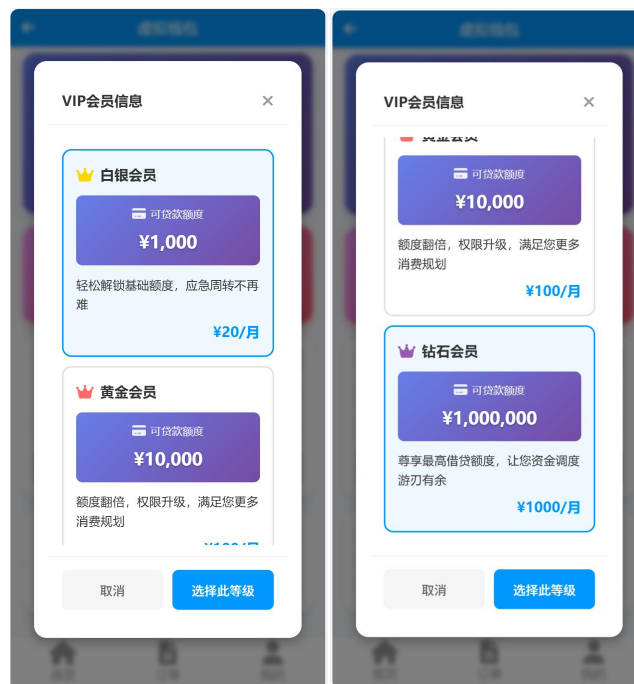


图 1-5 VIP会员信息

透支（贷款）：用户在消费过程中，当钱包可用余额不足以支付时，用户可启用透支额度完成即时消费。

免息期：享受 1 个月的免息期。

计息规则：超过 1 个月未还款的部分开始计息，日利率为 0.02%。

应还款项计算：应还款项 = 贷款本金 + (还款时间 - 贷款时间) 的月份差 $\times$ 0.02% $\times$ 贷款金额。

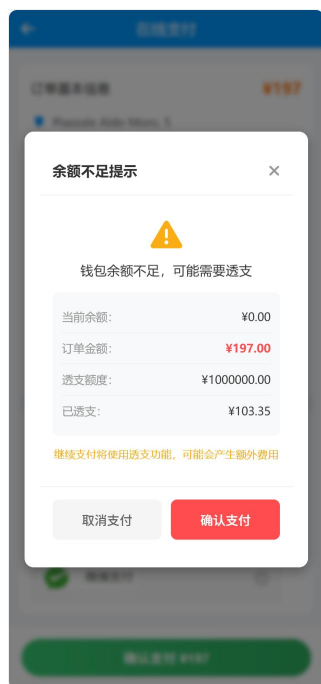


图 1-6 消费时的贷款提醒

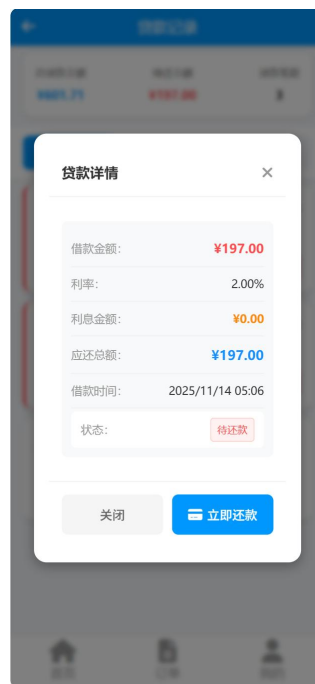


图1-7 具体的贷款详情

2.信息展示:

(1) 账户余额展示

实时余额：在虚拟钱包页面实时展示当前账户的可用余额。

操作入口：页面提供清晰的“充值”和“提现”操作入口。



图 1-8 虚拟钱包账户余额

(2) 可透支额度展示

信用信息：显示用户的当前会员等级、可透支总额度以及已使用的透支金额。

还款引导：设置“去还款”按钮，点击后跳转至贷款记录页面，便于用户管理信用负债。



图 1-9 透支信息展示图



图 1-10 交易明细状态管理

(3) 交易明细查询

查询维度：用户可输入时间范围对交易记录进行筛选。

交易类型：支持按类型筛选，包括：充值、提现、支付、收款、退款。

交易状态：支持按状态筛选，包括 正常（资金已完成流转，变为可用余额）、冻结（资金处于在途或担保状态，如买家确认收货前的货款），暂不可用



图 1-11 虚拟钱包交易明细

4. 贷款记录管理

总览信息： 页面顶部清晰展示总贷款金额、待还本息金额以及累计贷款笔数。

状态分类： 贷款记录可按状态查看：全部、待还款、已还款。

单笔详情： 对于每一笔贷款款项，必须展示以下具体信息：

借款时间： 实际发生透支或贷款的时间。

贷款金额： 贷款的原始本金金额。

已还金额： 当前已归还的本金和利息总额。

待还本息： 剩余需要归还的本金和利息总额。

借款利率： 当前适用的计息利率（如 0.02%）。



图 1-12 贷款记录页面



图 1-13 贷款详情

1.3 业务分析

1.3.1 业务分析类图

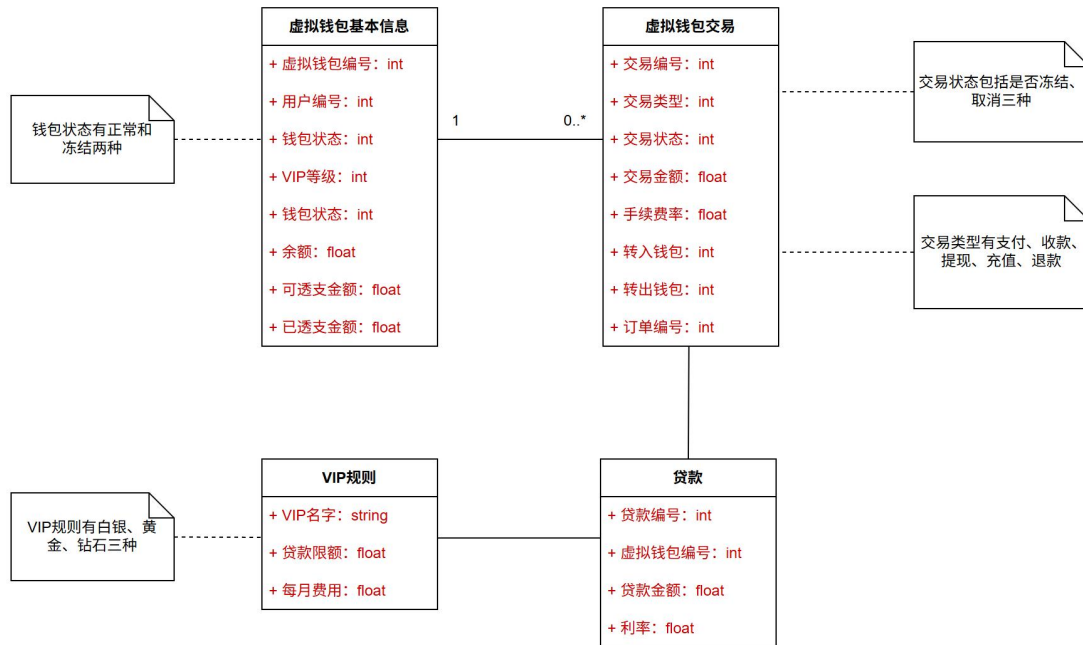


图 1-14 虚拟钱包类图

1.3.2 业务流程分析 (UML活动图)

(1) 虚拟钱包流程

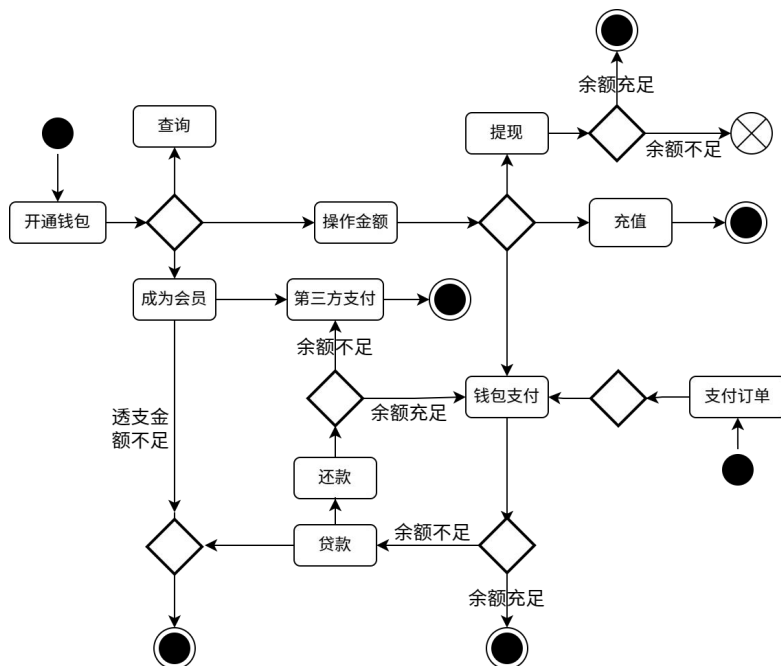


图 1-15 虚拟钱包活动图

1.3.3 电子钱包系统用例图

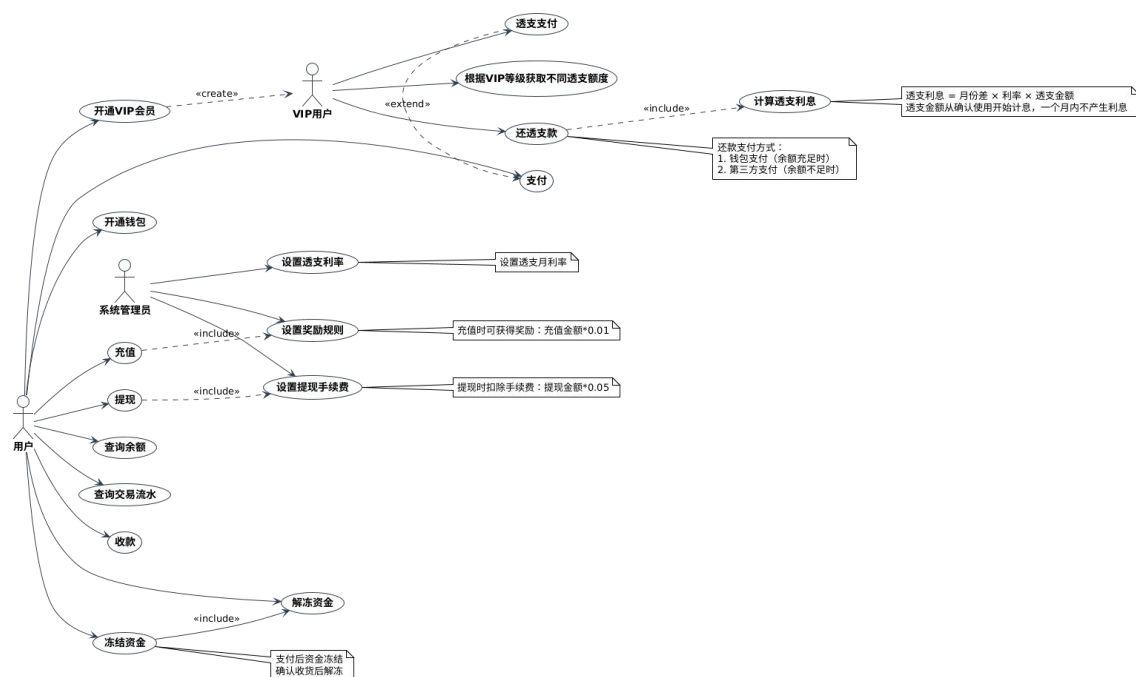


图 1-16 虚拟钱包用例图

表 1-2 开通钱包用例说明

编号	UC-001	名称	开通钱包
执行者	用户	优先级	高
描述	用户首次开通虚拟钱包功能		
前置条件	用户已注册饿了么账号且未开通钱包		
基本流程	1. 进入在线支付页面 2. 选择虚拟钱包支付 3. 确认开通虚拟钱包		
结束状况	钱包开通成功，初始余额为0		
可选流程	1.进入我的页面 2.点击“虚拟钱包” 3.点击“确认开通”		
异常流程	-		
备注	开通后即可使用充值、支付等功能		

表 1-3 充值用例说明

编号	UC-002	名称	充值
执行者	用户	优先级	高
描述	用户向虚拟钱包账户存入资金		
前置条件	用户已登录且钱包账户状态正常		
基本流程	1. 选择充值功能 2. 输入充值金额 3. 选择支付方式 4. 确认充值 5. 系统应用奖励规则 6. 更新钱包余额		
结束状况	充值成功，钱包余额增加		
可选流程	-		
异常流程	-		
备注	充值时可获得奖励：充值金额*0.01		

表 1-4 提现用例说明

编号	UC-003	名称	提现
执行者	用户	优先级	高
描述	用户从虚拟钱包提取资金到第三方支付系统		
前置条件	用户已登录且钱包有可用余额		
基本流程	1. 选择提现功能 2. 输入提现金额 3. 选择到账系统 4. 系统计算并扣除手续费 5. 确认提现 6. 更新钱包余额		
结束状况	提现成功，钱包余额减少		
可选流程	-		
异常流程	-		
备注	提现时扣除手续费：提现金额*0.05		

表 1-5 支付用例说明

编号	UC-004	名称	支付
执行者	用户	优先级	高
描述	用户使用钱包余额支付订单		
前置条件	用户已登录且有足够余额		
基本流程	1.进入在线支付页面 2.选择虚拟钱包支付 3.确认支付		
结束状况	支付成功，资金进入冻结状态		
可选流程	透支支付（VIP用户）		
异常流程	余额不足		
备注	支付后资金冻结，需确认收货后解冻		

表 1-6 查询余额用例说明

编号	UC-005	名称	查询余额
执行者	用户	优先级	高
描述	用户查看电子钱包的当前可用余额		
前置条件	用户已登录且钱包账户存在		
基本流程	1.进入虚拟钱包页面 2.显示当前可用余额		
结束状况	成功显示余额信息		
可选流程	-		
异常流程	-		
备注	-		

表 1-7 查询交易流水用例说明

编号	UC-006	名称	查询交易流水
执行者	用户	优先级	中
描述	用户查看钱包的所有交易记录		
前置条件	用户已登录且有交易历史		
基本流程	1.进入虚拟钱包页面 2.点击“交易明细” 3.查看详细交易流水		
结束状况	成功显示交易流水		
可选流程	选择开始日期和结束日期，查看特定时期的交易流水		
异常流程	-		
备注	记录包含充值、提现、支付、收款、退款等类型		

表 1-8 收款用例说明

编号	UC-007	名称	收款
执行者	用户	优先级	中
描述	商家用户接受来自顾客用户的付款		
前置条件	用户已登录且钱包状态正常		
基本流程	1.顾客用户确认收货 2.资金转入收款方钱包 3.生成收款记录		
结束状况	收款成功，钱包余额增加		
可选流程	-		
异常流程	-		
备注	-		

表 1-9 冻结资金用例说明

编号	UC-008	名称	冻结资金
执行者	系统（自动）	优先级	高
描述	系统在顾客支付订单但还没有确认收货时暂时冻结这笔付款		
前置条件	顾客支付订单还没有确认收货		
基本流程	1.顾客支付订单 2.系统锁定相应金额 3.更新顾客钱包余额		
结束状况	资金冻结成功		
可选流程	-		
异常流程	-		
备注	主要用于交易担保场景		

表 1-10 解冻资金用例说明

编号	UC-009	名称	解冻资金
执行者	系统（自动）	优先级	高
描述	系统在顾客确认收货后解冻这笔付款		
前置条件	存在已冻结的资金且满足解冻条件		
基本流程	1.顾客确认收货 2.系统释放冻结金额 3.更新对应商家钱包余额		
结束状况	资金打入对应商家钱包		
可选流程	-		
异常流程	-		
备注	-		

表 1-11 开通VIP会员用例说明

编号	UC-010	名称	开通VIP会员
执行者	用户	优先级	中
描述	普通用户开通VIP会员，享受透支特权		
前置条件	用户已开通钱包且账户状态正常		
基本流程	1.进入虚拟钱包页面 2.点击“去成为VIP” 3.选择VIP等级 4.支付开通费用		
结束状况	用户开通VIP会员，享受透支特权		
可选流程	-		
异常流程	-		
备注	不同VIP等级对应不同透支额度：白银会员可贷款额度为1000元，黄金会员可贷款额度为10000元，钻石会员可贷款额度为1000000元		

表 1-12 透支支付用例说明

编号	UC-011	名称	透支支付
执行者	VIP用户	优先级	高
描述	在钱包余额不足时使用透支额度完成支付		
前置条件	用户有可用透支额度且余额不足		
基本流程	1.进入在线支付页面 2.选择虚拟钱包支付 3.在弹出的“钱包余额不足，可能需要透支”警告框中点击“确认支付” 4.产生一条贷款记录		
结束状况	支付成功，产生贷款记录		
可选流程	-		
异常流程	-		
备注	透支金额从确认使用开始计息，一个月内不产生利息		

表 1-13 还透支款用例说明

编号	UC-012	名称	还透支款
执行者	VIP用户	优先级	高
描述	偿还已使用的透支金额及相应利息		
前置条件	用户有未偿还的透支金额		
基本流程	1.进入虚拟钱包页面 2.点击“去还款” 3.点击“还款”并选择还款方式（钱包余额、第三方支付） 4.确认还款		
结束状况	还款成功，透支金额清零		
可选流程	-		
异常流程	-		
备注	利息=月份差*利率*透支金额		

第二章 模块设计

2.1 数据库设计

2.1.1 DB 一览表（新增部分）

表 2-1 新增数据库表一览

NO	数据库表名	说明
1	virtual_wallet	虚拟钱包表
2	virtual_wallet_loan	虚拟钱包贷款表
3	virtual_wallet_transaction	虚拟钱包交易表
4	virtual_wallet_vip_rule	虚拟钱包VIP规则表
5	system_config	系统配置表

2.1.2 表结构

对于表约束，使用简写，说明如表2-2。

表 2-2 约束说明

标识	英文	解释
PK	primary key	主键
FK	foreign key	外键
NN	not null	非空
UQ	unique	唯一索引
AI	auto increment	自增长列
DEL	delete	逻辑删除标记 0-正常 1-删除

表 2-3 virtual_wallet 虚拟钱包表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	虚拟钱包id
2	user_id	bigint			NN	用户id
3	create_time	timestamp				创建时间
4	update_time	timestamp				更新时间
5	status	tinyint				钱包状态 0-正常 1-冻结
6	vip_level	tinyint				vip级别 0-非vip
7	is_deleted	tinyint				是否删除
8	balance	decimal	10	2	NN	余额
9	overdraft_amount	decimal	10	2	NN	可透支金额
13	overdrawn_amount	decimal	10	2	NN	已透支金额

表 2-4 virtual_wallet_loan 虚拟钱包贷款表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	虚拟钱包id
2	wallet_id	bigint			NN	用户id
3	loan_amount	decimal	10	2	NN	贷款金额
4	create_time	timestamp			NN	创建时间
5	repay_time	timestamp				还款时间
6	rate	decimal	10	2	NN	利息率/月

表 2-5 virtual_wallet_transaction 虚拟钱包交易表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	虚拟钱包交易id
2	type	tinyint				交易类型 0-支付 1-收款 2-提现 3-充值 4-退款
3	status	tinyint				操作金额是否为冻结 0-否 1-是 2-取消标记
4	amount	decimal	10	2	NN	操作金额
5	from_account	bigint			NN	转出钱包 交易类型为充值 时值为0
6	to_account	bigint			NN	转入钱包 交易类型为提现 时值为0
7	is_deleted	tinyint				是否删除
8	fee	decimal	10	2		手续费或奖励
9	fee_rate	decimal	10	2		手续费率或奖励率
10	create_time	timestamp			NN	创建时间
11	update_time	timestamp			NN	更新时间
12	order_id	bigint				若是订单支付的流水，关 联订单id，用于冻结操作

表 2-6 virtual_wallet_vip_rule 虚拟钱包VIP规则表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	虚拟钱包vip规则id
2	name	varchar	255		NN	vip名字
3	description	text				vip描述
4	is_deleted	tinyint				是否删除
5	overdraft_limit	decimal	10	2	NN	贷款限额
6	cost	decimal	10	2	NN	每月花费金额

表 2-7 system_config 系统配置表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	int			NN PK AI	配置id
2	config_key	varchar			NN	配置键
3	config_value	varchar			NN	配置值
4	value_type	varchar			NN	配置值类型
5	description	varchar				描述
6	update_time	timestamp			NN	更新时间

为了将原有系统的订单支付与钱包联动，orders订单表新添字段 payment_method，表示支付方式：0-微信支付 1-支付宝支付 2-虚拟钱包支付。

2.2 后端接口设计

为方便响应处理与异常情况的判断，统一将接口响应的数据模型统一为 JsonResult，具体说明如表2-8。

表 2-8 JsonResult 包装类字段说明

字段名	类型	说明
success	boolean	是否出现异常
code	string	异常状态码
data	object	返回数据
message	string	异常信息

2.2.1 新增接口

(1) 钱包信息相关接口

1.获取虚拟钱包手续费和奖励规则

/api/wallet/rule

请求方式：GET

参数：无

返回值：JsonResult<String>

2.充值

/api/wallet/recharge

请求方式：GET

参数：amount 充值金额

返回值：JsonResult<Boolean>

3.提现

/api/wallet/withdrawal

请求方式: GET

参数: amount 提现金额

返回值: `HttpResult<Boolean>`

4.提现/充值预览3

/api/wallet/preview

请求方式: GET

参数: amount 充值金额、option 0-充值 1-提现

返回值: `HttpResult<PreviewVO>` 手续费/奖励金额信息

5.用户开通钱包

/api/wallet/open

请求方式: GET

参数: 无

返回值: `HttpResult<Long>`, 开通的钱包ID

6.获取用户钱包信息

/api/wallet/message

请求方式: GET

参数: 无

返回值: `HttpResult<WalletVO>`

(2) VIP相关接口

1.获取虚拟钱包 VIP 规则

/api/wallet/vip/rule

请求方法: GET

参数: 无

返回值: `HttpResult<ListWalletVipVO>` 虚拟钱包VIP规则列表

2. 申请VIP

/api/wallet/vip/apply

请求方法: GET

参数: vipLevel vip等级

返回值: `HttpResult<Boolean>`

(3) 贷款相关接口

1. 还贷款

/api/wallet/loan/repay

请求方法: GET

参数: id 贷款记录id

返回值: `HttpResult<Boolean>`

2. 查看钱包贷款

/api/wallet/loan/list

请求方法: GET

参数: 无

返回值: `HttpResult<List<VirtualWalletLoan>>` 贷款记录列表

3. 查看钱包贷款详细信息

/api/wallet/loan/detail

请求方法: GET

参数: loanId 贷款记录ID

返回值: `HttpResultList<VirtualWalletLoan>`、贷款详细信息

(4) 钱包支付相关接口

1. 使用钱包支付订单

/api/wallet/transaction/payment

请求方式: GET

参数: orderId 订单id

返回值: `HttpResult<Boolean>`

2.获取用户钱包明细列表

/api/wallet/transaction/list

请求方式: GET

参数: type (交易类型: 0-支付 1-收款 2-提现 3-充值 4-退款)、status (交易状态: 操作金额是否为冻结 0-否 1-是 2-取消标记)、startDate (开始日期)、endDate (截止日期)

返回值: `HttpResult<List<TransactionRecordVO>>` 交易明细列表

3.根据明细id获取指定明细详细信息

/api/wallet/transaction/detail

请求方式: GET

参数: transactionId 交易id

返回值: `HttpResult<TransactionRecordDetailVO>` 交易明细详细信息

4.根据订单id获取指定明细详细信息

/api/wallet/transaction/detail/order

请求方式: GET

参数: orderId 订单id

返回值: `HttpResult<TransactionRecordDetailVO>` 交易明细详细信息

2.3 前端设计

1.钱包开通

虚拟钱包入口位于“我的”页面中, 见图2-1左1

- ① 自动检测: 系统自动检测用户是否已开通虚拟钱包
- ② 一键开通: 未开通用户可一键申请开通虚拟钱包
- ③ 初始化设置: 开通成功后自动初始化账户余额和基本信息



图 2-1 虚拟钱包入口



图 2-2 开通钱包



2.余额管理

- ① 余额显示: 实时显示当前账户余额
- ② 充值功能: 支持在线充值，包含充值奖励机制
- ③ 提现功能: 支持余额提现，自动计算手续费
- ④ 智能预览: 充值/提现时实时计算费用和到账金额

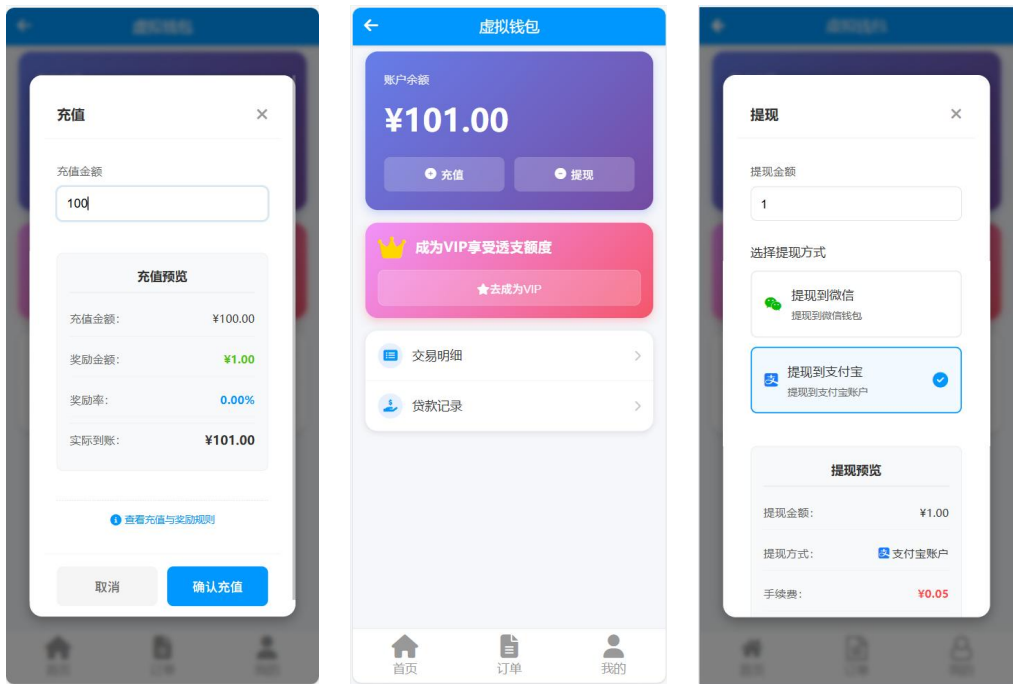


图 2-3 余额管理

3. VIP 会员

- ① 等级查看: 显示当前VIP等级和权益信息
- ② 等级升级: 支持多级VIP会员升级选择
- ③ 透支额度: 不同VIP等级享受不同的透支额度
- ④ 权益展示: 清晰展示各等级的具体权益和费用

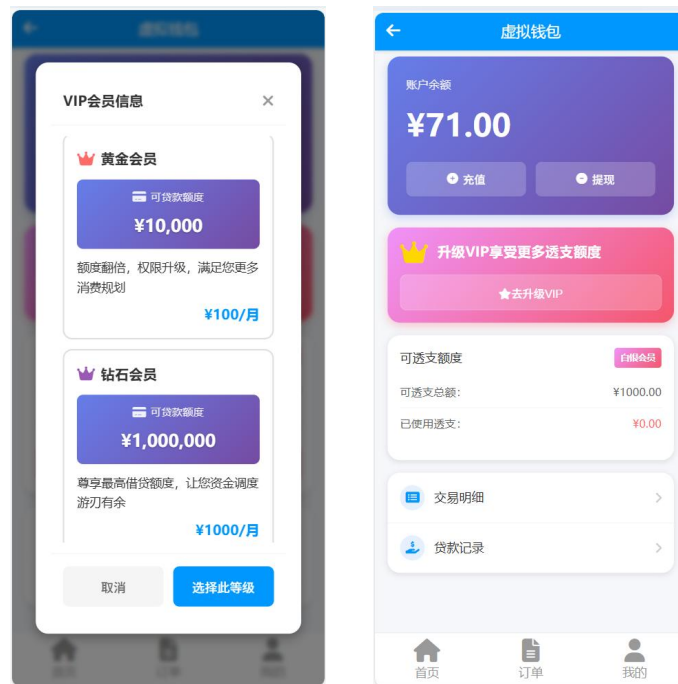


图 2-4 会员弹窗

4. 透支管理

- ① 额度查看: VIP用户可查看当前可用透支额度
- ② 使用记录: 显示已使用的透支金额
- ③ 额度提醒: 透支额度不足时的智能提醒



图 2-5 透支管理弹窗

5.交易记录

- ① 记录展示: 显示所有钱包交易记录（充值、提现、支付、收款、退款）
- ② 分类筛选: 按交易类型、状态、时间范围进行筛选
- ③ 统计分析: 显示总收入、总支出、交易笔数等统计信息
- ④ 详情查看: 查看完整的交易详细信息

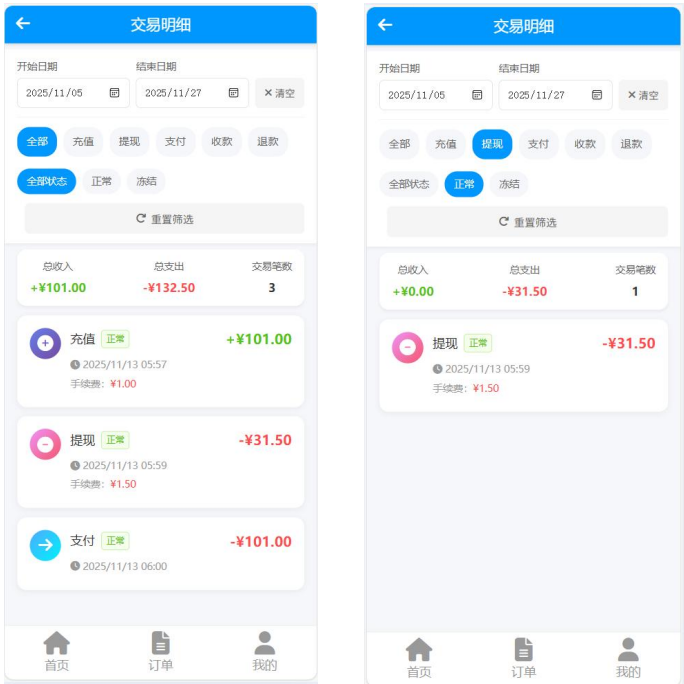


图 2-6 交易记录页面

6. 贷款管理

- ① 贷款记录: 显示所有贷款记录和状态
- ② 贷款详情: 展示贷款金额、利率、利息、还款时间等信息
- ③ 状态筛选: 按还款状态（待还款/已还款）筛选记录
- ④ 在线还款: 支持钱包余额或第三方支付方式还款



图 2-7 贷款管理页面

第三章 模块实现

3.1 模块概览

对于本次虚拟钱包的模块，使用充血模型，结合 Springboot 框架，整体模块架构见图3-1。与经典的贫血模型的三层架构不同 (controller-service-mapper)，充血模型主要引入邻域层 Domain，将对同种业务对象的操作聚合到一个邻域对象中，而非在 service 中实现所有的业务逻辑，以实现高内聚低耦合。

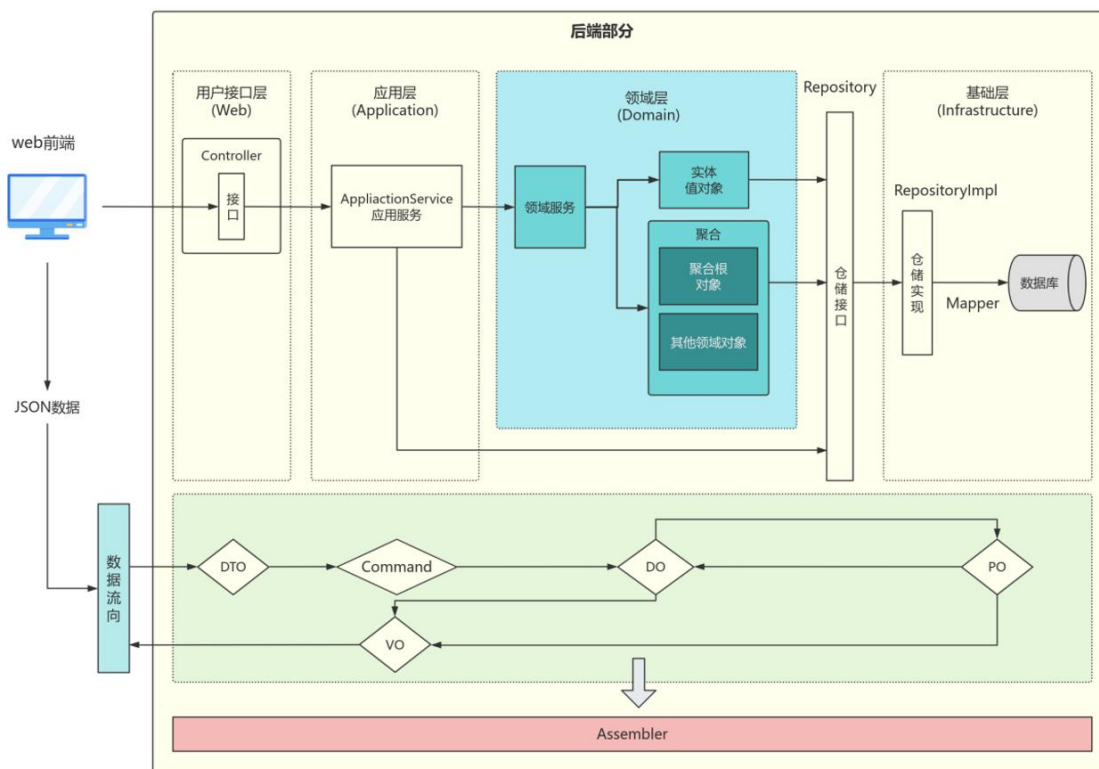


图 3-1 模块贫血框架

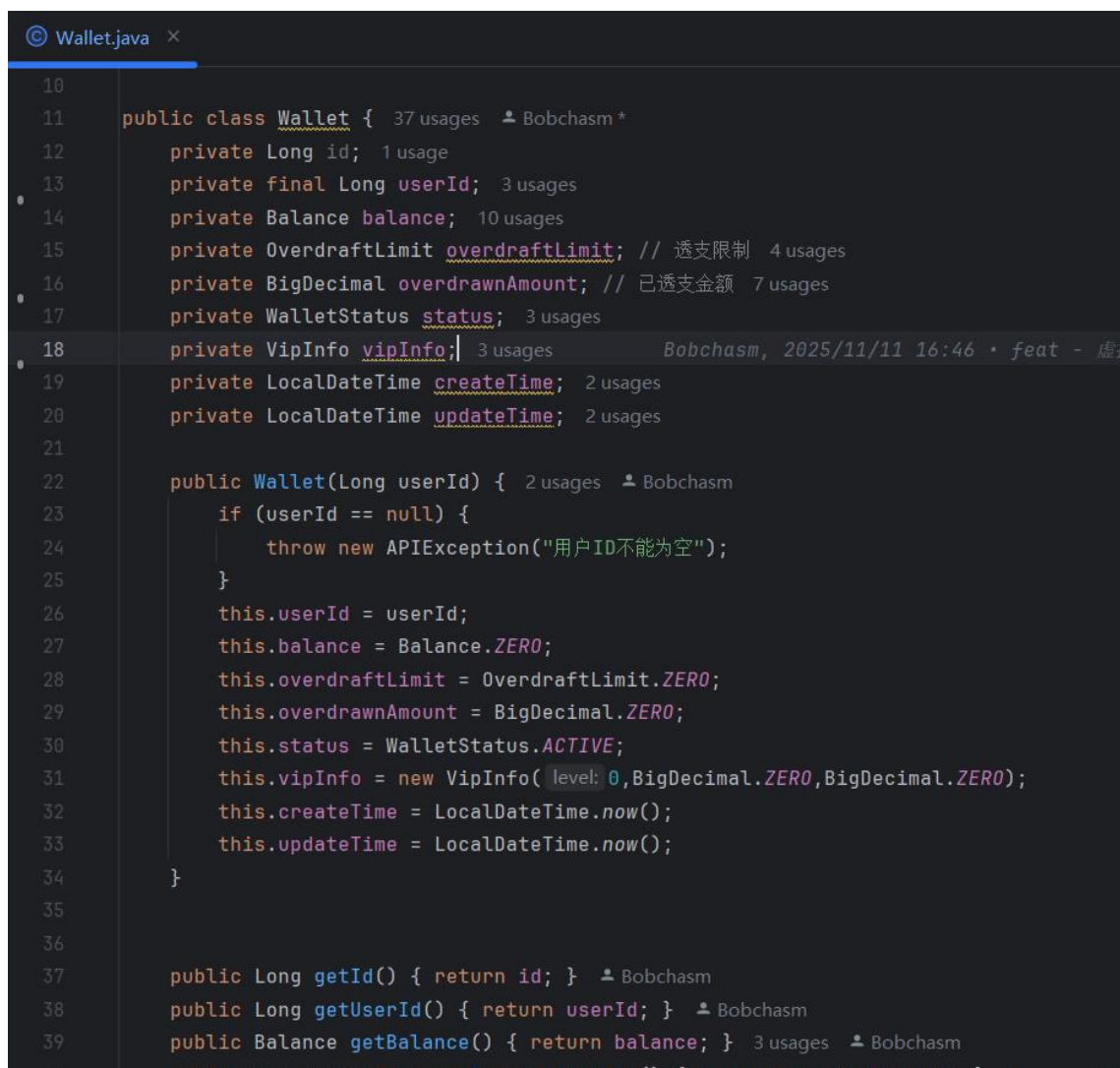
一个接口请求的处理流程也有较大的变化。前端发来请求，仍然由用户接口层 (Web) 的 controller 处理，框架将 Json 参数处理成后端能够方便使用的数据类型或者 DTO 对象；将请求数据传至应用层 (Application) 的 service，相较于贫血模型的 service，应用层的内容主要用于整合不同领域的处理，只是用于统筹各个领域而非包含具体的领域业务逻辑，在这一层中，通过对领域对 DO 的操作，将具体的逻辑写进其专属的领域中，并继续将数据传递至仓库接口层；于是进入到基础设施层，仓库接口接收到领域数据与操作后，根据其实现(面向抽象)将领域对象转换成与数据库能够使用的数据(主要是与数据库表对应的 PO)，调用 mapper 层的接口，与数据库交互。

在返回过程中，数据库返回的数据还不能直接用于业务操作。在本系统中，对于简单的查询(主要是不需要额外处理的查询接口)保留部分贫血思想，直接将 PO 或 VO 返回给应用层的 QueryService (可以抽象成经典贫血模型中的 service)，该 service

继续将数据传递给 **controller** 并返回给前端。这样的实现主要因为：简单查询几乎不涉及业务逻辑，不需要对领域对象进行操作，如使用完整的充血框架，数据多传递了层数且最终数据结构没有发生变化，但中间经历了许多不必要的类型转换，于是为了避免这种过度设计，在简单查询中简化层数，但又保持充血架构的完整性和一致性。

对于非简单查询的接口返回时，首先对于数据库有返回的PO对象，先在仓库实现中转换成领域对象，再传递给应用层，应用层通过对领域对象的处理，并整合不同领域间的协调，再将领域中的数据转换成最终接口返回的VO，传递至Web层，并最终响应。

领域是充血模型中最重要的概念，它将最相关的逻辑聚合到一个对象中，最终由应用层统筹。结合框架实现，领域对象是一个纯Java对象，不依赖框架，不含框架注解，也不存在依赖注入，在虚拟钱包模块中，最重要的领域是钱包聚合根 **Wallet**，见图3-2。领域中几乎不包含 **Setter** 方法，不仅包含属性，最重要的是包含操作其的方法。



```
10
11 public class Wallet { 37 usages  Bobchasm *
12     private Long id; 1 usage
13     private final Long userId; 3 usages
14     private Balance balance; 10 usages
15     private OverdraftLimit overdraftLimit; // 透支限制 4 usages
16     private BigDecimal overdrawnAmount; // 已透支金额 7 usages
17     private WalletStatus status; 3 usages
18     private VipInfo vipInfo; 3 usages Bobchasm, 2025/11/11 16:46 • feat - 虚拟
19     private LocalDateTime createTime; 2 usages
20     private LocalDateTime updateTime; 2 usages
21
22     public Wallet(Long userId) { 2 usages  Bobchasm
23         if (userId == null) {
24             throw new APIException("用户ID不能为空");
25         }
26         this.userId = userId;
27         this.balance = Balance.ZERO;
28         this.overdraftLimit = OverdraftLimit.ZERO;
29         this.overdrawnAmount = BigDecimal.ZERO;
30         this.status = WalletStatus.ACTIVE;
31         this.vipInfo = new VipInfo(level: 0, BigDecimal.ZERO, BigDecimal.ZERO);
32         this.createTime = LocalDateTime.now();
33         this.updateTime = LocalDateTime.now();
34     }
35
36
37     public Long getId() { return id; }  Bobchasm
38     public Long getUserId() { return userId; }  Bobchasm
39     public Balance getBalance() { return balance; } 3 usages  Bobchasm
```

图 3-2 Wallet领域

同时，数据在不同层的传递中，需要在不同的类型转换，具体见图？下方的数据流向。对于这些转换的操作主要编写在基础设施层的Assembler中，各个层中的对象转换可以调用该类的方法，图3-3展示钱包领域对象到数据库持久化对象的转换。

```
@Component 2 usages  Bobchasm
public class WalletAssembler {
    /**
     * Wallet领域对象 → VirtualWallet持久化对象
     */
    public VirtualWallet toPO(Wallet wallet) { 2 usages  Bobchasm
        if (wallet == null) return null;
        VirtualWallet po = new VirtualWallet();
        po.setId(wallet.getId());
        po.setUserId(wallet.getUserId());
        po.setBalance(wallet.getBalance().getAmount());
        po.setOverdraftAmount(wallet.getOverdraftLimit().getAmount());
        po.setOverdrawnAmount(wallet.getOverdrawnAmount());
        po.setStatus(wallet.getStatus().getCode());
        po.setVipLevel(wallet.getVipInfo().getLevel());
        po.setCreateTime(wallet.getCreateTime());
        po.setUpdateTime(wallet.getUpdateTime());
        return po;
    }
}
```

图 3-3 钱包Assembler

项目充血模块的目录结构见图3-4。

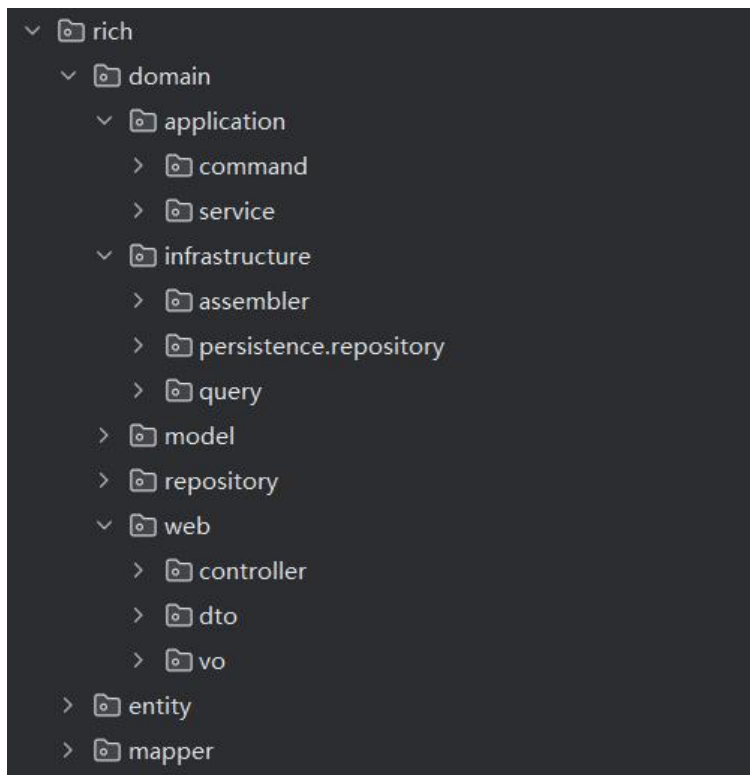


图 3-4 模块目录结构

3.2 业务逻辑实现

3.2.1 领域模型

1. Wallet 钱包聚合根

用于操作用户的钱包，主要包含钱包支出、收入、还贷以及校验钱包状态等逻辑方法，该领域是整个虚拟钱包系统的核心操作对象，其包含的属性见图3-5。

```
public class Wallet { 37 usages  👤 Bobchasm *
    private Long id; 1 usage
    private final Long userId; 3 usages
    private Balance balance; 10 usages
    private OverdraftLimit overdraftLimit; // 透支限制 4 usages
    private BigDecimal overdrawnAmount; // 已透支金额 7 usages
    private WalletStatus status; 3 usages
    private VipInfo vipInfo; 3 usages
    private LocalDateTime createTime; 2 usages
    private LocalDateTime updateTime; 2 usages
}
```

图 3-5 Wallet领域属性

2. Transaction 交易领域

主要用于操作收入与支出发生的交易，提供了不同的构造方法，便于业务操作。

3. Balance 金额领域

钱包的主要操作在于金额，这里主要包含金额扣减、增加、判断支出是否可支持等核心操作方法。

4. Loan 贷款领域

主要包含利息计算等方法。

5. OverdraftLimit 超支领域

6. VipInfo 会员信息领域

3.2.2 具体实现

1.钱包开通

用户尝试进入虚拟钱包时，首先调用接口 `/api/wallet/message` 获取钱包信息校验用户是否已经开通钱包。未开通时接口data为null。开通钱包将在用户钱包表中插入一条属于该用户的记录。同时在订单支付选择钱包支付时，同样会进行相同校验。应用核心实现见图3-6。


```
public Long open() { 1 usage  ▲ Bobchasm *
    User user = userMapper.findByUsernameWithAuthorities(SecurityUtils.getCurrentUsername().
        orElseThrow(() -> new APIException(ResultCodeEnum.VALUE_MISSED)));
    Wallet wallet = walletRepository.findById(user.getId());
    if (null != wallet) {
        throw new APIException(ResultCodeEnum.VIRTUAL_WALLET_OPENED);
    }
    wallet = new Wallet(user.getId());
    return walletRepository.createWallet(wallet);
}
```

图 3-6 开通钱包核心实现

2. 升级为会员

当支付时如果钱包余额不足，会员可以透支一定数值的金额，进行贷款。模块设置了不同等级的会员，其可透支的金额数量不同。钱包表中存在透支金额相关字段，成为会员时修改该字段，核心业务整合见图3-7。

```
public Boolean applyVip(Integer toVipLevel) { 1 usage  ▲ Bobchasm *
    User user = userMapper.findByUsernameWithAuthorities(SecurityUtils.getCurrentUsername().
        orElseThrow(() -> new APIException(ResultCodeEnum.VALUE_MISSED)));
    Wallet wallet = walletRepository.findById(user.getId());
    VipInfo vip = vipInfoRepository.findByLevel(toVipLevel);
    if (!wallet.compareVipLevel(vip)) {
        throw new APIException("您已经拥有该项vip的权益");
    }
    // You, Moments ago • Uncommitted changes
    setVipInfo(wallet, vip);
    wallet.upgrade(vip.getOverdraftLimit());
    walletRepository.modifyWallet(wallet);

    return true;
}
```

图 3-7 会员实现

3. 使用钱包支付订单

这是钱包的核心业务之一。先获取指定订单信息，由于订单部分仍然为贫血模型，应用层直接调用订单的 mapper 层接口获取，再通过订单获取到对应商家的用户，使用用户id获取收支两方的钱包信息，这里调用 repository 的接口，获取到的是二者钱包的领域模型，接着对领域模型进行操作。

各个领域业务整合见图3-9。

(1) 首先扣除顾客钱包的金额，调用顾客钱包Wallet领域的 pay() 方法(见图3-8)，利用Balance领域中的 canAfford() 校验当前余额与可透支金额是否充足。充足时其会返回需要贷款的金额，供应用层进一步处理。

```

public BigDecimal pay(BigDecimal amount) { 2 usages Bobchasm
    validateActiveStatus();
    BigDecimal totalAvailable = balance.getAmount()
        .add(overdraftLimit.getAmount())
        .subtract(overdrawnAmount);

    // 余额+可贷款不足
    if (totalAvailable.compareTo(amount) < 0) {
        throw new APIException(ResultCodeEnum.BALANCE_LIMIT);
    }

    // 优先使用余额
    BigDecimal balancePayment = balance.getAmount().min(amount);
    BigDecimal overdraftPayment = amount.subtract(balancePayment);
    if (balancePayment.compareTo(BigDecimal.ZERO) > 0) {
        balance = balance.subtract(balancePayment);
    }
    if (overdraftPayment.compareTo(BigDecimal.ZERO) > 0) {
        overdrawnAmount = overdrawnAmount.add(overdraftPayment);
    }

    return overdraftPayment;
}

```

图 3-8 领域 pay 方法

- (2) 根据上一步返回的贷款数值办理贷款，操作领域Loan，然后调用对应 repository 的贷款接口，在数据库中插入一条记录。
- (3) 创建Transaction流水领域对象，将该笔交易的数据添加进数据库中，这里将该流水的status设置为冻结，即商家还未真正收到金额。
- (4) 更新订单状态为已支付，直接调用订单的 mapper 接口。

```

// 顾客出账
BigDecimal loadAmount = fromWallet.pay(order.getOrderTotal());
walletRepository.modifyWallet(fromWallet);

// 是否需要贷款，办理贷款
if (!loadAmount.equals(BigDecimal.ZERO)) {
    loanRepository.load(fromWallet.getId(), loadAmount, LOAD_RATE);
}

// 交易，此时商家账户还未进账，金额暂留在交易中
Transaction transaction = new Transaction(TransactionType.PAYMENT, order.getOrderTotal(),
    fromWallet.getId(), toWallet.getId(), BigDecimal.ZERO, status: 1);

transactionRepository.payOrder(transaction, orderId);
// 设置订单已支付状态
ordersMapper.setOrderState(orderId, orderState: 1);
// 设置支付方式
ordersMapper.setOrderPaymentMethod(orderId, method: 2);

```

图 3-9 支付订单业务实现

4.金额流转

当订单被确认完成后，冻结在交易中的金额被商家接收，通过商家用户钱包的领域对象的collection方法操作，之后再进行 repository 操作，并利用transaction更新该笔交易的状态。

```
public void collection(BigDecimal amount) { 3 usages
    validateActiveStatus();
    balance = balance.add(amount);
}
```

图 3-10 领域收款方法

同时对于充值和提现，同样利用钱包聚合根中的 pay() 与 collection() 方法，其同样会插入transaction流水记录，充值时付款方设为0，提现时收款方设为0，0表示抽象的第三方平台。

5.贷款

在订单支付时超支可根据会员金额设定进行贷款，用户查看贷款时可以获取到利息，该数据使用Loan领域的 countInterest() 计算。

```
public BigDecimal countInterest() { 2 usages  Bobchasm
    long month;
    if(null != repayTime) {
        month = ChronoUnit.MONTHS.between(loanDate.toLocalDate(), repayTime.toLocalDate());
    }else{
        LocalDate now = LocalDate.now();
        month = ChronoUnit.MONTHS.between(loanDate.toLocalDate(),now);
    }

    return loanAmount.multiply(BigDecimal.valueOf(month * loanInterestRate)); Bobchasm, T
}
```

图 3-11 领域 countInterest() 方法

第四章 模块测试

4.1 测试概览

本次测试旨在验证饿了么项目中虚拟钱包功能的正确性、稳定性和安全性，确保系统满足需求文档中的所有功能要求。

4.1.1 测试工具

- Apifox（API测试）
- Chrome DevTools（前端调试）
- Vue DevTools（Vue调试）

4.1.2 测试范围

- 前端功能测试
- 后端API测试
- 前后端联合测试
- 边界条件测试
- 异常场景测试

4.2 前端测试

4.2.1 页面加载与渲染测试

1. 钱包主页面加载（Wallet.vue）

表 4-1 主页面加载测试

测试项	测试步骤	预期结果	测试结果	备注
页面初始加载	打开钱包页面	显示加载动画，加载完成后显示钱包信息	通过	加载速度正常
余额显示	查看余额卡片	正确显示用户余额，格式为¥XX.XX	通过	数值精确到小数点后2位
VIP信息显示	非VIP用户查看	显示“去成为VIP”按钮和VIP推广卡片	通过	-
VIP信息显示	VIP用户查看	显示VIP等级、透支额度、已使用透支	通过	VIP徽章正确显示
功能入口显示	查看功能区	正确显示“交易明细”和“贷款记录”入口	通过	图标和文字清晰

2. 交易明细页面（WalletTransactions.vue）

表 4-2 交易明细页面测试

测试项	测试步骤	预期结果	测试结果	备注
明细列表加载	打开交易明细页面	按时间倒序显示所有交易记录	通过	-
交易类型筛选	选择不同交易类型	正确筛选显示对应类型的记录	通过	支持充值、提现、支付等
交易状态筛选	选择不同状态	正确筛选显示对应状态的记录	通过	支持成功、失败、处理中
日期范围筛选	选择日期范围	显示指定日期范围内的交易	通过	日期选择器正常工作
交易详情查看	点击某条交易记录	弹窗显示详细的交易信息	通过	信息完整准确

3. 贷款记录页面（WalletLoan.vue）

表 4-3 贷款记录页面测试

测试项	测试步骤	预期结果	测试结果	备注
贷款列表显示	打开贷款记录页面	显示所有贷款记录，区分已还和未还	通过	-
还款操作	点击还款按钮	弹出还款确认框，显示还款金额	通过	-
还款方式选择	选择还款方式	可选择钱包支付或第三方支付	通过	两种方式均可用

4.2.2 充值功能测试

表 4-4 充值功能测试

测试项	测试步骤	预期结果	测试结果	备注
打开充值弹窗	点击“充值”按钮	弹出充值模态框	通过	-
支付方式选择	选择微信/支付宝	选中状态正确显示，有视觉反馈	通过	默认选中微信
充值金额输入	输入充值金额	实时显示充值预览信息	通过	包括奖励金额和到账金额
充值预览计算	输入100元	自动计算并显示奖励（假设10%奖励率）	通过	计算准确

无效金额验证	输入0或负数	确认按钮置灰，无法提交	通过	-
空金额验证	不输入金额点击确认	提示“请输入有效的充值金额”	通过	toast提示清晰
充值规则查看	点击规则链接	弹出规则说明弹窗	通过	规则内容完整
充值成功提示	充值成功后	显示成功提示，包含支付方式信息	通过	“充值成功！已通过微信支付充值”
关闭弹窗	点击取消或关闭按钮	弹窗关闭，数据重置	通过	-

4.2.3 提现功能测试

表 4-5 提现功能测试

测试项	测试步骤	预期结果	测试结果	备注
打开提现弹窗	点击“提现”按钮	弹出提现模态框	通过	-
提现方式选择	选择微信/支付宝	选中状态正确显示，有视觉反馈	通过	默认选中微信
提现金额输入	输入提现金额	实时显示提现预览信息	通过	包括手续费和到账金额
提现预览计算	输入100元	自动计算并显示手续费	通过	计算准确，显示余额
余额不足验证	输入超过余额的金额	预览中显示警告信息	通过	红色警告文字
无效金额验证	输入0或负数	确认按钮置灰，无法提交	通过	-
提现成功提示	提现成功后	显示成功提示，包含提现方式信息	通过	“提现成功！将提现至微信钱包”

4.2.4 VIP功能测试

表 4-6 VIP功能测试

测试项	测试步骤	预期结果	测试结果	备注
VIP等级列表	点击“去成为VIP”	显示所有VIP等级及对应权益	通过	-
VIP等级选择	点击某个VIP等级	卡片高亮显示，勾选图标显示	通过	-
贷款额度显示	查看VIP卡片	清晰显示每个等级的可贷款额度	通过	金额格式化显示
VIP支付确认	选择等级后点击确认	显示支付确认弹窗，包含费用和贷款额度	通过	-
VIP升级成功	支付成功后	刷新钱包信息，显示新VIP等级	通过	透支额度立即生效

4.3 后端测试

使用 Apifox 进行接口测试。

4.3.1 钱包管理API测试

1. 开通钱包 (GET /api/wallet/open)

表 4-7 开通钱包接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常开通	有效token	返回钱包ID，初始余额为0	返回200，data为钱包ID	通过
重复开通	已有钱包的用户	返回错误提示	返回400，提示已存在	通过
未登录开通	无token	返回401未授权	返回401	通过

2. 获取钱包信息 (GET /api/wallet/message)

表 4-8 获取钱包信息接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常获取	有效token	返回完整钱包信息	返回余额、VIP信息、透支额度等	通过
未开通获取	未开通钱包的用户	返回404错误	返回404，提示钱包不存在	通过
数据完整性	-	包含所有必要字段	userId, balance, vipLevel, overdraftAmount等	通过

4.3.2 充值与提现API测试

1. 充值 (GET /api/wallet/recharge)

表 4-9 充值接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常充值	amount=100	余额增加100+奖励, 返回true	余额正确增加	通过
小额充值	amount=0.01	成功充值最小金额	充值成功	通过
无效金额	amount=0	返回错误提示	返回400, 提示金额无效	通过
负数金额	amount=-100	返回错误提示	返回400	通过
交易记录	充值后查询	生成充值类型的交易记录	记录正确生成	通过

2. 提现 (GET /api/wallet/withdrawal)

表 4-10 提现接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常提现	amount=100	余额减少100+手续费, 返回true	余额正确减少	通过
余额不足	amount超过余额	返回错误提示	返回400, 提示余额不足	通过
提现0元	amount=0	返回错误提示	返回400	通过
手续费计算	amount=100	扣除的总金额=100+手续费	计算正确	通过
交易记录	提现后查询	生成提现类型的交易记录	记录正确生成	通过

3. 预览 (GET /api/wallet/preview)

表 4-11 预览接口测试

测试项	请求参数	预期结果	实际结果	测试结果
充值预览	amount=100, option=0	返回奖励金额和总到账金额	返回正确的预览数据	通过
提现预览	amount=100, option=1	返回手续费和实际到账金额	返回正确的预览数据	通过
各种金额	测试不同金额	计算公式正确应用	所有金额计算正确	通过

4.3.3 VIP功能API测试

1. 获取VIP规则 (GET /api/wallet/vip/rule)

表 4-12 获取VIP规则接口测试

测试项	请求参数	预期结果	实际结果	测试结果
获取规则列表	-	返回所有VIP等级及对应权益	返回完整VIP列表	通过
数据完整性	-	包含等级、费用、透支额度等	所有字段完整	通过

2. 申请VIP (GET /api/wallet/vip/apply)

表 4-13 申请VIP接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常申请	vipLevel=1	扣除费用，升级VIP，增加透支额度	升级成功	通过
余额不足	余额不足以支付VIP费用	返回错误提示	返回400，提示余额不足	通过
重复申请	申请已有等级	返回提示	处理正确	通过
无效等级	vipLevel=999	返回错误提示	返回400	通过

4.3.4 交易记录API测试

1. 获取交易列表 (GET /api/wallet/transaction/list)

表 4-14 获取交易列表接口测试

测试项	请求参数	预期结果	实际结果	测试结果
获取全部记录	无筛选参数	返回用户所有交易记录	返回完整列表	通过
类型筛选	type=0（充值）	只返回充值记录	筛选正确	通过
状态筛选	status=1（成功）	只返回成功的交易	筛选正确	通过
日期范围	startDate, endDate	返回指定日期范围的记录	筛选正确	通过
组合筛选	多个筛选条件	正确应用所有条件	筛选正确	通过

2. 获取交易详情 (GET /api/wallet/transaction/detail)

表 4-15 获取交易详情接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常查询	transactionId=1	返回该交易的详细信息	返回完整详情	通过
无效ID	transactionId=999999	返回404或空结果	返回404	通过
他人交易	查询其他用户的交易ID	返回403或404	拒绝访问	通过

4.3.5 贷款功能API测试

1. 获取贷款列表 (GET /api/wallet/loan/list)

表 4-16 获取贷款列表接口测试

测试项	请求参数	预期结果	实际结果	测试结果
获取全部贷款	-	返回用户所有贷款记录	返回完整列表	通过
区分已还未还	-	repayTime字段标识是否已还	字段正确	通过

2. 还款 (GET /api/wallet/loan/repay)

表 4-17 还款接口测试

测试项	请求参数	预期结果	实际结果	测试结果
钱包还款	id=1, option=1	从钱包余额扣除, 更新贷款状态	还款成功	通过
第三方还款	id=1, option=0	模拟第三方支付, 更新贷款状态	还款成功	通过
余额不足	钱包余额不足还款	返回错误提示	返回400	通过
重复还款	已还的贷款再次还款	返回错误提示	返回400	通过

4.3.6 订单支付API测试

1. 钱包支付订单 (GET /api/wallet/transaction/payment)

表 4-18 支付订单接口测试

测试项	请求参数	预期结果	实际结果	测试结果
正常支付	orderId=1	扣除订单金额，生成交易记录	支付成功	通过
余额不足	订单金额>余额	使用透支额度或返回余额不足	处理正确	通过
使用透支	VIP用户余额不足	使用透支额度支付	透支成功	通过
无效订单	orderId=999999	返回404错误	返回404	通过
他人订单	支付他人订单	返回403错误	拒绝访问	通过

4.4 前后端联合测试

4.4.1 充值流程完整测试

表 4-19 充值流程完整测试

步骤	操作	预期结果	实际结果	测试结果
1	打开钱包页面	显示当前余额	余额显示正常	通过
2	点击充值按钮	弹出充值弹窗	弹窗正常显示	通过
3	选择支付方式（微信）	微信选项高亮	选中状态正确	通过
4	输入充值金额100	实时显示预览信息	显示奖励1元，到账101元	通过
5	点击确认充值	调用后端API	请求成功发送	通过
6	后端处理	余额增加，生成交易记录	处理成功	通过
7	前端接收响应	显示成功提示	显示“充值成功！已通过微信支付充值”	通过
8	刷新钱包信息	余额更新为新金额	余额正确更新	通过
9	查看交易明细	显示刚才的充值记录	记录正确显示	通过

测试结论：充值流程完整，前后端数据同步正确。

4.4.2 提现流程完整测试

表 4-20 提现流程完整测试

步骤	操作	预期结果	实际结果	测试结果
1	打开钱包页面	显示当前余额101元	余额显示正常	通过
2	点击提现按钮	弹出提现弹窗	弹窗正常显示	通过
3	选择提现方式（支付宝）	支付宝选项高亮	选中状态正确	通过
4	输入提现金额50	实时显示预览信息	显示手续费2.5元，到账47.5元	通过
5	点击确认提现	调用后端API	请求成功发送	通过
6	后端处理	余额减少50元，生成交易记录	处理成功	通过
7	前端接收响应	显示成功提示	显示“提现成功！将提现至支付宝账户”	通过
8	刷新钱包信息	余额更新为50元	余额正确更新	通过
9	查看交易明细	显示刚才的提现记录	记录正确显示	通过

测试结论：提现流程完整，手续费计算正确，前后端数据同步准确。

4.4.3 VIP升级流程完整测试

表 4-21 VIP升级流程完整测试

步骤	操作	预期结果	实际结果	测试结果
1	点击“去成为VIP”	显示VIP等级列表	列表正确显示	通过
2	查看白银会员信息	显示费用20元/月，贷款额度1000元	信息正确显示	通过
3	选择白银会员	卡片高亮显示	选中状态正确	通过
4	点击确认	显示支付确认弹窗	弹窗正确显示费用和额度	通过
5	确认支付	调用后端API	请求成功发送	通过
6	后端处理	扣除20元（第三方支付系统处理），升级VIP，增加透支额度	处理成功	通过
7	前端接收响应	显示成功提示	显示“VIP升级成功！”	通过
8	刷新钱包信息	显示VIP徽章和透支额度	VIP信息正确显示	通过
9	验证透支功能	支付订单时可使用透支	透支功能正常	通过

测试结论：VIP升级流程完整，透支额度立即生效。

4.4.4 订单支付流程完整测试

表 4-22 订单支付流程完整测试

步骤	操作	预期结果	实际结果	测试结果
1	创建订单（金额30元）	订单创建成功	订单正常创建	通过
2	选择钱包支付	跳转到支付页面	页面正常跳转	通过
3	确认支付	调用钱包支付API	请求成功发送	通过
4	后端扣款	从钱包余额扣除30元	扣款成功	通过
5	生成交易记录	创建支付类型的交易记录	记录正确生成	通过
6	前端跳转	跳转到支付成功页面	页面正确跳转	通过
7	查看钱包	余额减少30元	余额正确更新	通过
8	查看交易明细	显示订单支付记录	记录显示完整的订单信息	通过

测试结论：钱包支付订单流程完整，与订单系统集成正常。

4.4.5 透支与还款流程测试

表 4-23 透支与还款流程测试

步骤	操作	预期结果	实际结果	测试结果
1	余额25元，透支额度500元	钱包信息正确显示	显示正常	通过
2	支付60元订单	余额不足，使用透支	支付成功	通过
3	检查钱包状态	余额0元，已透支35元	状态正确	通过
4	去还款	待还款记录消失，增加一条已还款记录，已使用透支额度归零	符合预期	通过
5	查看贷款记录	显示自动还款记录	记录正确	通过

测试结论：透支和还款逻辑正确。

4.5 边界条件测试

4.5.1 数值边界测试

表 4-24 数值边界测试

测试项	输入值	预期结果	实际结果	测试结果
最小充值金额	0.01元	充值成功	成功	通过
最小提现金额	0.01元	提现成功	成功	通过
超大金额充值	999999.99元	充值成功	成功	通过
精度测试	0.123元	四舍五入到0.12元	正确处理	通过
负数金额	-100元	拒绝	正确拒绝	通过
零金额	0元	拒绝	正确拒绝	通过

4.5.2 并发测试

表 4-25 并发测试

测试项	测试场景	预期结果	实际结果	测试结果
同时充值	同一用户两次充值请求	两次都成功，金额累加正确	处理正确	通过
同时提现	同一用户两次提现请求	按顺序处理，余额计算正确	处理正确	通过
充值提现同时	同时发起充值和提现	按顺序处理，最终余额正确	处理正确	通过

4.6 异常场景测试

4.6.1 权限测试

表 4-26 权限测试

测试项	测试场景	预期结果	实际结果	测试结果
未登录访问	未登录打开钱包	跳转到登录页	正确跳转	通过
Token过期	Token过期时操作	提示重新登录	提示正确	通过
跨用户访问	访问他人交易记录	拒绝访问	正确拒绝	通过

4.6.2 数据一致性测试

表 4-27 数据一致性测试

测试项	测试场景	预期结果	实际结果	测试结果
刷新页面	操作后立即刷新	数据保持一致	一致	通过
多标签同步	两个标签页同时操作	数据正确同步	同步正常	通过
离线缓存	网络断开后查看数据	显示上次缓存的数据	缓存正确	通过

第五章 问题与解决&结果反思

5.1 问题与解决

下面主要列出一下较为重要的问题，在实现过程的debug中还遇到许多细节的问题与bug，主要与编写失误与逻辑疏忽有关，不一一列出。

1.提现业务逻辑问题

首次接口编写时提现金额校验逻辑为对比本金加手续费与余额的数值，见图？，而现实中提现扣款为输入金额，而手续费包含在扣款中而非多扣的金额

解决：修改校验逻辑，只返回要扣掉的手续费，这部分费用对钱包本身的金额本质上没有影响，只对用户起到一个提醒作用。直接注释该校验代码，同时钱包扣除的金额即为用户输入金额

```
BigDecimal fee = amount.multiply(BigDecimal.valueOf(WITHDRAWAL_RATE));

if (!wallet.getBalance().canAfford(amount.add(fee))) {
    throw new APIException(ResultCodeEnum.BALANCE_LIMIT.getMessage());
}

wallet.pay(amount.add(fee));
walletRepository.modifyWallet(wallet);
```

图 5-1 提现业务逻辑问题

2.领域对象属性赋值问题

测试接口时，Wallet中的createTime一直设置失败

解决：Wallet领域中createTime误设为final，导致repository层将PO转换成Domain时由于通过反射在对象创建后尝试设置该值时失败，于是取消该属性的final设置

```
public class Wallet { 37 usages  Bobchasm
    private Long id; 1 usage
    private final Long userId; 3 usages

    private Balance balance; 10 usages
    private OverdraftLimit overdraftLimit; 4
    private BigDecimal overdrawnAmount; // 已

    private WalletStatus status; 3 usages
    private VipInfo vipInfo; 3 usages

    private final LocalDateTime createTime;
    private LocalDateTime updateTime; 2 usages

public Wallet toDomain(VirtualWallet po) { 1 usage  Bobchasm
    if (po == null) return null;

    Wallet wallet = new Wallet(po.getUserId());
    setId(wallet, po.getId());
    setField(wallet, fieldName: "balance", new Balance(po.getBalance()));
    setField(wallet, fieldName: "overdraftLimit", new OverdraftLimit(po.getOverdraftLimit()));
    setField(wallet, fieldName: "overdrawnAmount", po.getOverdrawnAmount());
    setField(wallet, fieldName: "status", WalletStatus.fromCode(po.getStatus()));
    setField(wallet, fieldName: "vipInfo", new VipInfo(po.getVipInfo()));
    setField(wallet, fieldName: "createTime", po.getCreateTime());
    setField(wallet, fieldName: "updateTime", po.getUpdateTime());
    return wallet;
}
```

图 5-2 领域对象属性赋值问题

5.2 结果反思

在模块实现的代码过程中，尽量避免一次性写很多的功能接口，而要完成一小部分时及时进行测试，使得小问题能及时解决，否则随着代码量和逻辑联系的增加，识别问题所在的难度和时间也会增加，更容易出现大问题。

同时在实现时要多注意字段、属性等名称的对应，往往一些细小的问题会出现在名称错误上而非代码本身问题。在数据库操作中，删改操作一定要保证WHERE子句的正确性，否则会造成难以恢复的错误。

对于本次充血模型框架的实践，开始实现的过程中没有把握好各种业务逻辑实现位置，经过整体梳理和项目模块充血框架整理，逐渐将各个领域专属的逻辑移动至领域内，提高聚合度，习惯通过领域来处理逻辑，应用层只用来整合各个领域的逻辑。同时在设计的过程中，需要灵活应对，不能一味的套用充血模型，而要考虑具体的功能逻辑，尽量避免过度设计。

第六章 实践总结与展望

6.1 实践总结

本次综合实践在完成前后端分离（SpringBoot + VUE）项目基础流程的基础上，成功集成了核心金融模块——虚拟钱包功能¹。相较于第一版依赖外部支付的模式，我们实现了平台内部的资金闭环管理，显著提升了系统的业务复杂度和技术深度。主要技术突破与学习体会如下：

(1) DDD充血模型实践

首次尝试在Service层引入领域驱动设计(DDD)的充血模型来构建钱包核心领域。

我们将充值奖励计算、提现手续费扣除、透支利息计算等复杂的金融业务逻辑内聚到钱包实体对象中，而非分散在服务方法中。

这一实践深刻理解了充血模型带来的优势：极大地增强了代码的内聚性和领域逻辑的一致性，避免了传统贫血模型中业务逻辑在 Service 层过度膨胀的问题，为高复杂度的金融业务逻辑提供了清晰且稳健的实现基础。

(2) 金融交易的原子性保障

在实现钱包对钱包的支付功能时，我们重点关注了交易原子性的保证。通过事务控制，确保了资金在付款方扣除和收款方增加的操作要么同时成功，要么同时失败，这是金融类系统稳定运行的基石。

(3) 复杂业务逻辑与风控实现

成功实现了充值返点、提现奖励金扣除、资金冻结/解冻机制以及VIP用户透支（小额贷款）和利息计算等高级功能。这些机制的实现，让我们深入理解了电商平台如何通过可配置规则实现资金沉淀激励和风险控制。

6.2 未来展望

本次实践虽已完成核心业务功能链的构建，但面对未来大规模用户带来的高并发挑战，系统仍需深度优化。下一阶段，我们将着重提升性能和架构的健壮性：通过引入压力测试来识别瓶颈，并利用 Redis（用于缓存与资金操作安全）和 消息队列（MQ）（用于异步处理和提高 QPS 等并发指标）等经典中间件来优化系统性能。同时，我们将探索如微服务等架构演进方案，对虚拟钱包等核心模块进行解耦，使系统更贴近实际生产应用。此外，还将通过限制多端登录等措施进一步提高系统安全性和容错性，确保系统在高负载下稳定运行，持续向贴近现实应用的复杂需求标准迈进。

¹ 仓库链接: <https://gitee.com/dai-mingjing/frontend-comprehension.git>